Sumario

¿Qué es JSON? 5
Características principales 5
Por qué se inventó 5
Usos principales de JSON 6
Analogía para entenderlo 6
Resumen 6
Historia y origen de JSON (comparación con XML) 7
Origen de JSON 7
Reconocimiento oficial 7
XML vs JSON: la competencia 7
Ejemplo comparativo 7
Ventajas de JSON sobre XML 8
Por qué no desapareció XML? 8
Línea de tiempo resumida 8
Resumen 8
Estructura básica de JSON (objetos y arreglos) 9
Introducción 9
1. Objetos en JSON 9
2. Arreglos en JSON 9
3. Objetos dentro de arreglos 10
4. Arreglos dentro de objetos 10
5. Anidación de objetos 10
6. Ejemplo de estructura mixta completa 11
Resumen 11
Tipos de datos en JSON 11
1. Strings (cadenas de texto) 12
2. Numbers (números) 12
3. Booleans (valores lógicos) 12
4. Null (valor nulo) 13
5. Arrays (arreglos o listas) 13
6. Objects (objetos o diccionarios) 13
Resumen gráfico de los tipos JSON 13
Resumen final 14
Reglas de sintaxis en JSON (comillas, llaves, corchetes, comas) 14
1. Comillas dobles obligatorias 14
2. Llaves { } para objetos 14
3. Corchetes [ ] para arreglos 15
4. Separación con comas 15
5. Claves y valores 15
6. Sensibilidad a mayúsculas y minúsculas 15
7. Espacios en blanco 16
8. Comentarios 16
9. Documento válido único 16
Resumen de reglas 17
Validación de un archivo JSON (herramientas y linters) 17
¿Qué significa validar un JSON? 17

Ejemplo de JSON inválido....17
Herramientas para validar JSON....18
1. Validadores en línea....18
2. Editores de código (con linters)....18
3. Consola y herramientas de línea de comandos....18
4. Validación con JSON Schema....19
Buenas prácticas al validar JSON....19
Resumen....19
Ventajas y desventajas de JSON....20
Ventajas de JSON....20
1. Simplicidad y legibilidad....20
2. Ligero y eficiente....20
3. Universal....20
4. Integración natural con JavaScript....20
5. Estructuras anidadas....21
6. Estándar ampliamente adoptado....21
Desventajas de JSON....21
1. Tipos de datos limitados....21
2. No permite comentarios....21
3. Redundancia en datos muy grandes....21
4. Menos seguro si no se valida....22
5. No ideal para configuraciones muy complejas....22
Comparación rápida con otros formatos....22
Resumen....22
JSON vs otros formatos (XML, YAML, CSV)....22
📌 JSON (JavaScript Object Notation)....22
📌 XML (eXtensible Markup Language)....23
📌 YAML (YAML Ain't Markup Language)....23
📌 CSV (Comma-Separated Values)....24
📊 Comparación directa....24
¿Cuándo usar cada uno?....25
Resumen....25
Lectura y escritura de JSON en JavaScript....25
¿Cómo maneja JSON JavaScript?....25
1. Convertir un JSON a objeto con JSON.parse()....25
2. Convertir un objeto a JSON con JSON.stringify()....26
3. Ejemplo práctico: Enviar JSON con fetch....26
4. Ejemplo práctico en navegador (localStorage)....27
5. Manejo de errores en JSON.parse()....27
6. Formateo y legibilidad con JSON.stringify()....27
📊 Resumen....28
Lectura y escritura de JSON en Python (módulo json)....28
1. Importar el módulo json....28
2. Convertir JSON a diccionario (json.loads)....28
3. Convertir diccionario a JSON (json.dumps)....29
4. Formatear JSON con indent y sort_keys....29
5. Leer JSON desde un archivo (json.load)....29
6. Escribir JSON en un archivo (json.dump)....30

7. Manejo de errores al parsear JSON....30
8. Conversión entre tipos JSON ↔ Python....30
📊 Resumen....31
Lectura y escritura de JSON en Java (biblioteca Jackson / Gson)....31
Opción 1: Usando Jackson....31
1) Dependencia (Maven)....31
2) Crear una clase Java (POJO)....32
3) Serializar objeto → JSON....32
4) Deserializar JSON → objeto....32
5) Leer y escribir JSON en archivo....33
Opción 2: Usando Gson (Google)....33
1) Dependencia (Maven)....33
2) Serializar objeto → JSON....33
3) Deserializar JSON → objeto....34
Comparación Jackson vs Gson....34
Resumen....34
Uso de JSON en APIs REST (request y response en formato JSON)....35
1) Concepto básico....35
2) Estructura de un request típico (HTTP)....35
3) Estructura de un response típico (HTTP)....35
4) Métodos y semántica....36
5) Ejemplos de consumo (cliente)....36
a) curl (CLI)....36
b) JavaScript (fetch)....36
c) Python (requests)....37
d) C# (.NET HttpClient)....37
6) Diseño de respuestas JSON....37
a) Recurso simple....37
b) Colección con paginación y filtros....37
7) Errores en JSON (formato recomendado)....38
8) Autenticación y autorización (headers)....38
9) CORS (navegadores)....38
10) Versionado de API....38
11) Caching (mejor rendimiento)....38
12) Idempotencia y seguridad....39
13) PATCH (actualización parcial)....39
14) Validación con JSON Schema (servidor/cliente)....39
15) Estándares de formato de respuesta....39
16) Seguridad y robustez....39
17) Mini API de ejemplo (contrato)....39
18) Checklist de buenas prácticas....40
Resumen....41
JSON Schema: validación y definición de datos....41
1) ¿Qué es JSON Schema?....41
2) Ejemplo simple....41
3) Elementos básicos de JSON Schema....42
4) Ejemplo con validaciones....42
5) Validar arrays....42

6) Objetos anidados....43
7) Ejemplo avanzado: catálogo de productos....44
8) Validación en distintos lenguajes....44
Python (con jsonschema)....44
JavaScript (con ajv)....45
9) Usos prácticos de JSON Schema....45
Resumen....45
Uso de JSON en bases de datos NoSQL (ejemplo: MongoDB)....46
1) Introducción....46
2) Documentos JSON en MongoDB....46
3) Inserción de documentos JSON....46
4) Consultas con JSON....47
5) Actualización con JSON....47
6) Eliminación de documentos....47
7) Ejemplo de estructura compleja....47
8) Validación de JSON en MongoDB....48
9) Conexión desde lenguajes de programación....48
Python (PyMongo)....48
Node.js (Driver oficial)....49
10) Ventajas de usar JSON en MongoDB....49
Resumen....49
Buenas prácticas al trabajar con JSON....50
1) Nombres de claves....50
2) Tipos de datos correctos....50
3) null vs campo ausente....50
4) Estructura y profundidad....50
5) Tamaño, paginación y filtros (APIs)....50
6) Envoltorios de respuesta....50
7) Errores estructurados....51
8) Versionado y compatibilidad....51
9) Validación con JSON Schema....51
10) Serialización estable....51
11) Seguridad....51
12) Internacionalización y texto....51
13) Comentarios y documentación....51
14) Actualizaciones parciales....52
15) Rendimiento....52
16) Ejemplo de contrato sano....52
17) Checklist rápido....52
18) Antipatrones comunes (y cómo corregirlos)....53
19) Herramientas útiles....53

¿Qué es JSON?

Definición

JSON significa JavaScript Object Notation (Notación de Objetos de JavaScript). Es un formato de texto ligero para el intercambio de datos, fácil de leer y escribir tanto para humanos como para máquinas.

Se utiliza principalmente para transmitir información entre un servidor y una aplicación web, aunque su uso se ha extendido a prácticamente todos los lenguajes de programación modernos.

Ejemplo simple de un objeto JSON:

```
{
  "nombre": "Juan",
  "edad": 30,
  "casado": false
}
```

Características principales

•Basado en texto — Los datos están en formato de texto plano, lo que permite enviarlos por internet fácilmente.

•Estructurado — Usa pares clave-valor ("clave": valor) y arreglos para organizar la información.

•Ligero — Más simple y menos verboso que otros formatos como XML.

•Universal — Aunque su nombre viene de JavaScript, hoy lo usan Python, Java, C#, PHP, Go, Ruby, Swift, entre muchos otros.

•Interoperable — Es el formato estándar para la mayoría de las APIs REST y aplicaciones modernas.

Por qué se inventó

Antes de JSON, el formato más usado para intercambio de datos era XML. El problema de es que es muy verboso, con muchas etiquetas de apertura y cierre, lo que hacía que los archivos fueran más grandes y difíciles de leer.

Ejemplo de los mismos datos en XML:

```
<persona>
  <nombre>Juan</nombre>
  <edad>30</edad>
  <casado>false</casado>
</persona>
```

En JSON, la misma información es más corta y clara:

```
{
  "nombre": "Juan",
```

```
  "edad": 30,
  "casado": false
}
```

Usos principales de JSON

•Comunicación cliente-servidor: enviar datos de un backend (servidor) a un frontend (web o app).

•Almacenamiento: guardar configuraciones, preferencias o bases de datos pequeñas.

•APIs REST: la mayoría de servicios modernos (Google, Twitter, GitHub, etc.) devuelven datos en JSON.

•Bases de datos NoSQL: como MongoDB o CouchDB, que usan JSON como formato nativo de almacenamiento.

Analogía para entenderlo

Podés pensar en JSON como una libreta de direcciones digital.

•Cada persona sería un objeto ({}).

•Cada dato (nombre, edad, dirección) sería un par clave-valor ("clave": valor).

•Y si tenés muchas personas, las guardás en una lista o arreglo ([]).

Ejemplo:

```
[
  {
    "nombre": "Ana",
    "edad": 25
  },
  {
    "nombre": "Luis",
    "edad": 40
  }
]
```

Resumen

•JSON es un formato de texto, ligero, estructurado y universal.

•Nació en JavaScript, pero hoy lo usan todos los lenguajes de programación.

•Es el estándar de facto para el intercambio de datos en la web.

•Ventaja: simple, legible y eficiente comparado con XML.

Historia y origen de JSON (comparación con XML)

Origen de JSON

Año: JSON surgió a principios de los años 2000.
Creador: Fue Douglas Crockford, un programador y empresario estadounidense que trabajaba con JavaScript en ese momento.
Necesidad: En la época, el formato XML era el estándar para intercambio de datos en aplicaciones web, pero era pesado, verboso y complicado de procesar.
Idea clave: Crockford observó que la sintaxis de objetos de JavaScript era simple y fácil de entender. A partir de esa idea, definió un estándar para representar datos de manera más clara y ligera.
En 2001, Crockford comenzó a promover JSON como alternativa a XML para la comunicación en la web.

Reconocimiento oficial

En 2006, JSON fue estandarizado por ECMA (ECMA-404: The JSON Data Interchange Format).
En 2013, se publicó también como estándar por la IETF (RFC 7159).
Hoy, JSON es el formato más usado en APIs REST (REST), microservicios (microservicios) y aplicaciones web y móviles.

XML vs JSON: la competencia

Durante muchos años, XML fue el formato dominante. Era utilizado en servicios web, configuraciones y hasta en documentos de oficina (ejemplo: Word y Excel usan internamente XML).
Pero JSON fue ganando terreno rápidamente gracias a su simplicidad y eficiencia.

Ejemplo comparativo

XML

```
<producto>
  <codigo>101</codigo>
  <descripcion>Teclado mecánico</descripcion>
  <precio>1200.50</precio>
  <disponible>true</disponible>
</producto>
```

JSON

```
{
  "codigo": 101,
  "descripcion": "Teclado mecánico",
  "precio": 1200.50,
  "disponible": true
}
```

📌 Con JSON se necesita menos texto, es más legible, y se adapta de forma natural a la programación orientada a objetos.

Ventajas de JSON sobre XML

•Más ligero — Ocupa menos espacio en disco y menos ancho de banda al transmitirse.
•Más rápido — Los navegadores y lenguajes lo interpretan más fácilmente.
•Legibilidad — Los humanos pueden leerlo con mayor claridad.
•Integración natural con JavaScript — Mientras que XML requiere parseadores complejos, JSON se puede usar directamente con JSON.parse() o JSON.stringify().

Por qué no desapareció XML?

Aunque JSON domina la web moderna, XML sigue existiendo en ciertos contextos:
•Documentos estructurados (ejemplo: SVG, MathML, Office Open XML).
•Comunicaciones en sistemas legados (ejemplo: SOAP).
•Estándares con validación estricta mediante XSD (XML Schema).
Sin embargo, en la mayoría de aplicaciones modernas y APIs, JSON reemplazó casi por completo a XML.

Línea de tiempo resumida

•1990s — XML es adoptado como estándar para intercambio de datos.
•2001 — Douglas Crockford propone JSON como alternativa más ligera.
•2006 — JSON es estandarizado por ECMA.
•2013 — JSON se convierte en RFC (Internet Standard).
•Hoy — JSON es el formato universal para APIs y aplicaciones web/móviles.

Resumen

•JSON nació para resolver las limitaciones de XML.
•Creado por Douglas Crockford en los años 2000.
•Fue adoptado rápidamente porque es más simple, ligero y natural para la programación moderna.
•Hoy es el estándar en casi todos los lenguajes y frameworks.

Estructura básica de JSON (objetos y arreglos)

Introducción

JSON organiza la información en dos estructuras fundamentales:

•Objetos: colecciones de pares clave-valor encerrados entre llaves `{}`.

•Arreglos (arrays): listas ordenadas de valores encerrados entre corchetes `[]`.

Estas dos formas se pueden combinar y anidar para representar datos complejos.

1. Objetos en JSON

Un objeto JSON es una colección de pares clave-valor.

•La clave siempre es una cadena de texto entre comillas dobles `" "`.

•El valor puede ser: string, number, boolean, null, otro objeto o un arreglo.

Ejemplo de un objeto JSON simple

```
{
  "nombre": "María",
  "edad": 28,
  "profesion": "Diseñadora",
  "activo": true
}
```

En este ejemplo:

•`"nombre"` es la clave — `"María"` es el valor.

•`"edad"` es la clave — `28` es el valor.

•`"profesion"` es la clave — `"Diseñadora"` es el valor.

•`"activo"` es la clave — `true` es el valor.

2. Arreglos en JSON

Un arreglo JSON es una lista ordenada de valores, encerrada entre `[]`. Los elementos se separan con comas.

Ejemplo de un arreglo JSON

```
[
  "Argentina",
  "Chile",
  "Uruguay",
  "Paraguay"
]
```

En este ejemplo, el arreglo contiene cuatro strings.

3. Objetos dentro de arreglos

Es muy común que un arreglo contenga varios objetos. Esto se utiliza, por ejemplo, para representar una lista de usuarios o productos.

Ejemplo: lista de personas

```
[
  {
    "nombre": "Ana",
    "edad": 25
  },
  {
    "nombre": "Luis",
    "edad": 40
  },
  {
    "nombre": "Sofía",
    "edad": 32
  }
]
```

Aquí tenemos un arreglo de tres objetos. Cada objeto representa a una persona con las claves `"nombre"` y `"edad"`.

4. Arreglos dentro de objetos

Un objeto puede tener dentro un arreglo como valor.

Ejemplo: una persona con lista de hobbies

```
{
  "nombre": "Carlos",
  "edad": 35,
  "hobbies": ["fútbol", "ajedrez", "cine"]
}
```

Aquí, `"hobbies"` es una clave, y su valor es un arreglo de strings.

5. Anidación de objetos

Los objetos también pueden contener otros objetos dentro.

Ejemplo: persona con dirección anidada

```
{
  "nombre": "Laura",
  "edad": 29,
  "direccion": {
    "calle": "Av. Colón",
    "numero": 742,
    "ciudad": "Córdoba"
  }
}
```

Aquí: `"direccion"` es una clave y su valor es otro objeto JSON con tres pares clave-valor.

6. Ejemplo de estructura mixta completa

Los objetos y arreglos se pueden combinar de forma compleja.

Ejemplo: catálogo de productos

```
{
  "tienda": "Tech Store",
  "productos": [
    {
      "codigo": 101,
      "descripcion": "Teclado mecánico",
      "precio": 1200.50,
      "disponible": true
    },
    {
      "codigo": 102,
      "descripcion": "Mouse inalámbrico",
      "precio": 850.75,
      "disponible": false
    }
  ],
  "ubicaciones": ["Buenos Aires", "Córdoba", "Rosario"]
}
```

En este JSON:

•"tienda" es un string.

•"productos" es un arreglo de dos objetos, cada uno con varios pares clave-valor.

•"ubicaciones" es un arreglo de strings.

Resumen

•JSON se construye con objetos {} y arreglos [].

•Los objetos almacenan información en pares clave-valor.

•Los arreglos almacenan listas ordenadas de elementos.

•Se pueden anidar objetos y arreglos para representar estructuras complejas.

•Esta flexibilidad hace que JSON sea ideal para representar datos del mundo real (usuarios, productos, pedidos, etc.).

Tipos de datos en JSON

JSON se diseñó para ser ligero y universal, por eso maneja un conjunto reducido de tipos de datos. Estos tipos son suficientes para representar la mayoría de la información que necesitamos en aplicaciones reales.

Los tipos de datos válidos en JSON son:

•Strings (cadenas de texto)
•Numbers (números)
•Booleans (valores lógicos)
•Null (valor nulo o vacío)
•Arrays (arreglos o listas)
•Objects (objetos o diccionarios)

1. Strings (cadenas de texto)

•Se escriben entre comillas dobles `" "`.
•Pueden contener letras, números, espacios y caracteres especiales.
•Se permiten caracteres Unicode, lo que significa que podés usar emojis o tildes.
Ejemplo:

```
{
  "nombre": "María",
  "ciudad": "Córdoba",
  "emoji": "😊"
}
```

Incorrecto en JSON: `'texto'` (comillas simples) — JSON no lo permite.

2. Numbers (números)

•Pueden ser enteros o decimales (con punto, no coma).
•No llevan comillas.
•Soportan signo positivo o negativo.
Ejemplo:

```
{
  "edad": 30,
  "altura": 1.75,
  "saldo": -1500.25
}
```

Importante: JSON no distingue entre enteros y decimales como algunos lenguajes (por ejemplo, Python sí distingue).

3. Booleans (valores lógicos)

•Se escriben en minúsculas: `true` o `false`.
•Se usan para representar estados lógicos o de activación.
Ejemplo:

```
{
  "activo": true,
  "admin": false
}
```

No se puede usar `True` o `False` con mayúscula (como en Python).

4. Null (valor nulo)

•Representa la ausencia de valor.

•Se escribe como `null` (todo en minúsculas).

Ejemplo:

```
{
  "telefono": null,
  "direccion": "Sin especificar"
}
```

Esto indica que la persona no tiene un número de teléfono cargado.

5. Arrays (arreglos o listas)

•Son listas ordenadas de valores.

•Se escriben entre corchetes `[]`.

•Pueden contener cualquier tipo de datos: strings, números, objetos, incluso otros arrays.

Ejemplo:

```
{
  "colores": ["rojo", "verde", "azul"],
  "numeros": [10, 20, 30, 40],
  "mixto": ["texto", 99, true, null]
}
```

6. Objects (objetos o diccionarios)

•Son colecciones de pares clave-valor.

•Se escriben entre llaves `{}`.

•Las claves siempre son strings entre comillas dobles.

•Los valores pueden ser cualquier tipo JSON.

Ejemplo:

```
{
  "producto": {
    "codigo": 101,
    "descripcion": "Monitor 24 pulgadas",
    "precio": 150.75,
    "disponible": true,
    "tags": ["tecnología", "pantalla", "oferta"]
  }
}
```

Aquí vemos un objeto con un arreglo dentro, mostrando la capacidad de anidación de JSON.

Resumen gráfico de los tipos JSON

Tipo — Ejemplo válido

String — "Hola mundo"

Number — 42, 3.14, -10

Boolean — true, false

Null — null

Array — ["a", "b", "c"]

Object — {"clave": "valor", "numero": 10}

Resumen final

•JSON maneja 6 tipos básicos de datos.

•No existen tipos más avanzados (fecha, hora, binarios) — se representan como strings.

•La simplicidad de este sistema hace que JSON sea fácil de parsear y universal.

Reglas de sintaxis en JSON (comillas, llaves, corchetes, comas)

JSON es un formato muy estricto en su sintaxis. A diferencia de otros lenguajes que a veces perdonan errores, en JSON cualquier detalle incorrecto rompe la validez del documento.

Aquí veremos las reglas fundamentales de escritura que todo archivo JSON debe cumplir.

1. Comillas dobles obligatorias

•Las claves siempre van entre comillas dobles `" "`.

•Los strings también deben usar comillas dobles, nunca simples.

📌 Correcto:

```
{
  "nombre": "Juan"
}
```

Incorrecto (comillas simples):

```
{
  'nombre': 'Juan'
}
```

2. Llaves { } para objetos

•Los objetos se encierran entre llaves `{ }`.

•Dentro de un objeto, se definen pares clave-valor.

📌 Ejemplo:

```
{
  "codigo": 101,
  "descripcion": "Teclado mecánico"
}
```

3. Corchetes [ ] para arreglos

•Los arreglos se encierran entre corchetes `[ ]`.

•Sus elementos se separan con comas.

📌 Ejemplo:

```
{
  "frutas": ["manzana", "banana", "pera"]
}
```

4. Separación con comas

•Los pares clave-valor en un objeto se separan con comas.

•Los elementos de un array también se separan con comas.

⚠️ Error común: colocar una coma al final del último elemento.

📌 Correcto:

```
{
  "ciudad": "Córdoba",
  "pais": "Argentina"
}
```

Incorrecto (coma extra):

```
{
  "ciudad": "Córdoba",
  "pais": "Argentina",
}
```

5. Claves y valores

•Las claves siempre son strings entre comillas dobles.

•Los valores pueden ser: string, número, booleano, null, objeto o array.

📌 Ejemplo:

```
{
  "activo": true,
  "saldo": 250.75,
  "amigos": ["Ana", "Luis"]
}
```

6. Sensibilidad a mayúsculas y minúsculas

•JSON es case-sensitive.

•`"true"` no es lo mismo que `"True"`.

•`"Nombre"` no es lo mismo que `"nombre"`.

📌 Ejemplo válido:

```
{
  "activo": true
}
```

Ejemplo inválido (boolean mal escrito):

```
{
  "activo": True
}
```

7. Espacios en blanco

•Los espacios, saltos de línea y tabulaciones son permitidos en JSON.

•No afectan al contenido, solo mejoran la legibilidad.

📌 Ejemplo válido (una línea):

```
{"nombre":"Pedro","edad":25}
```

📌 Ejemplo válido (con formato):

```
{
  "nombre": "Pedro",
  "edad": 25
}
```

8. Comentarios

Importante: JSON NO permite comentarios. Cualquier intento de escribir `//` o `/* ... */` hará que el JSON sea inválido.

Incorrecto:

```
{
  "nombre": "Ana" // nombre de la persona
}
```

Si querés agregar anotaciones, se suele usar un campo adicional:

📌 Correcto (técnica alternativa):

```
{
  "nombre": "Ana",
  "_comentario": "Este campo representa el nombre de la persona"
}
```

9. Documento válido único

•Un archivo JSON válido debe tener un solo objeto o un solo array en la raíz.

•No puede haber varios objetos sueltos.

📌 Correcto:

```
{
  "usuario": "Carlos",
  "edad": 33
}
```

📌 También válido:

```
[
  {"usuario": "Ana"},
```

```
  {"usuario": "Pedro"}
]
```

Incorrecto:

```
{ "usuario": "Ana" }
{ "usuario": "Pedro" }
```

Resumen de reglas

•Claves y strings — comillas dobles.
•Objetos — entre llaves `{}`.
•Arreglos — entre corchetes `[]`.
•Elementos separados con comas (sin coma final).
•Valores válidos: string, number, boolean, null, objeto, array.
•JSON distingue mayúsculas y minúsculas.
•Se permiten espacios y saltos de línea.
•No se permiten comentarios.
•Solo un objeto o array raíz.

Validación de un archivo JSON (herramientas y linters)

¿Qué significa validar un JSON?

Validar un archivo JSON significa comprobar que cumple estrictamente con las reglas de sintaxis. Un JSON válido debe:
•Estar correctamente formateado.
•Tener llaves `{}` y corchetes `[]` balanceados.
•No contener comas de más.
•Usar comillas dobles en claves y strings.
•Tener un único objeto o array en la raíz.
Si el archivo contiene cualquier error de sintaxis, deja de ser válido y las aplicaciones que intenten leerlo generarán un error de parsing.

Ejemplo de JSON inválido

```
{
  "nombre": "Ana",
  "edad": 28,
  "activo": True,   //  Error: True debe ser true y no se permiten
comentarios
```

```
}
```

Errores detectados:

•"True" con mayúscula — JSON solo acepta true.

•Coma final después de true.

•Comentario // — JSON no lo permite.

Herramientas para validar JSON

1. Validadores en línea

Son la forma más rápida de comprobar si un JSON está bien escrito. Algunos de los más usados:

•JSONLint — Valida y formatea JSON.

•JSON Formatter & Validator — Valida y muestra errores de forma detallada.

•Code Beautify JSON Validator.

📌 Funcionamiento:

•Pegás tu JSON en el cuadro de texto.

•La herramienta te dice si es válido o te marca los errores.

2. Editores de código (con linters)

Hoy casi todos los editores tienen soporte nativo para JSON.

•Visual Studio Code (VS Code) — Resalta errores de sintaxis en tiempo real y permite instalar extensiones como Prettier para formateo automático.

•Sublime Text — Con plugins que validan y formatean JSON.

•JetBrains (PyCharm, WebStorm, IntelliJ) — Validación y autocompletado para JSON y JSON Schema.

3. Consola y herramientas de línea de comandos

En Linux/macOS con jq:

jq es un procesador de JSON muy popular.

📌 Ejemplo:

```
jq . archivo.json
```

•Si el archivo es válido — muestra el JSON formateado.

•Si es inválido — indica en qué línea está el error.

En Python con el módulo json:

```
import json

with open("archivo.json", "r", encoding="utf-8") as f:
    try:
        data = json.load(f)
```

```
        print("El JSON es válido")
    except json.JSONDecodeError as e:
        print("Error en el JSON:", e)
```

En Node.js:

```
node -e "JSON.parse(require('fs').readFileSync('archivo.json'))"
```

•Si el JSON es válido, no muestra nada.

•Si es inválido, lanza un error.

4. Validación con JSON Schema

Más allá de la sintaxis, a veces necesitamos verificar que un JSON siga un formato esperado. Para eso se usa JSON Schema, un estándar que define reglas sobre:

•Tipos de datos esperados.

•Campos obligatorios.

•Valores permitidos.

📌 Ejemplo de JSON Schema (estructura de un producto):

```
{
  "type": "object",
  "properties": {
    "codigo": { "type": "integer" },
    "descripcion": { "type": "string" },
    "precio": { "type": "number" }
  },
  "required": ["codigo", "descripcion"]
}
```

Este esquema define que:

•codigo debe ser entero.

•descripcion debe ser string.

•precio debe ser número.

•codigo y descripcion son obligatorios.

Buenas prácticas al validar JSON

•Siempre usar comillas dobles en claves y strings.

•Evitar comas finales en arrays u objetos.

•Usar linters en el editor para detectar errores en tiempo real.

•Validar con herramientas online o CLI antes de usar en producción.

•Si es para APIs — definir un JSON Schema para mayor control.

Resumen

•Validar JSON = comprobar que cumple con la sintaxis estricta.

•Hay validadores online, linters en editores y herramientas en consola (como jq, Python o Node.js).

•Para verificar estructura y tipos — usar JSON Schema.
•Una validación correcta garantiza que el JSON será legible y portable en cualquier lenguaje.

Ventajas y desventajas de JSON

JSON se ha convertido en el formato estándar de intercambio de datos en aplicaciones modernas. Su popularidad se debe a ventajas claras frente a otros formatos como XML o YAML, aunque también tiene limitaciones que conviene conocer.

Ventajas de JSON

1. Simplicidad y legibilidad

JSON tiene una sintaxis muy fácil de leer y escribir tanto para humanos como para máquinas.
Ejemplo:

```
{
  "usuario": "Ana",
  "edad": 28,
  "activo": true
}
```

Es mucho más breve y claro que un equivalente en XML.

2. Ligero y eficiente

•Ocupa menos espacio que XML, ya que no usa etiquetas de apertura/cierre.
•Requiere menos ancho de banda al transmitirse por internet, clave en APIs y servicios web.

3. Universal

•Aunque nació en el mundo de JavaScript, hoy está soportado en casi todos los lenguajes de programación (Python, Java, C#, Go, PHP, etc.).
•Se puede serializar y deserializar fácilmente en cualquier tecnología moderna.

4. Integración natural con JavaScript

En JavaScript, un JSON se puede convertir en objeto con una sola instrucción:

```
let datos = JSON.parse('{"usuario": "Ana", "edad": 28}');
```

Esto lo hizo ideal para el auge de las aplicaciones web dinámicas (AJAX).

5. Estructuras anidadas

JSON permite representar estructuras complejas: objetos dentro de objetos, arrays dentro de arrays, etc.

Ejemplo:

```
{
  "curso": "Programación",
  "alumnos": [
    {"nombre": "Juan", "nota": 8},
    {"nombre": "Sofía", "nota": 10}
  ]
}
```

6. Estándar ampliamente adoptado

•JSON está definido por ECMA-404 y RFC 8259.

•Se convirtió en el formato predeterminado en APIs REST y microservicios.

•La mayoría de servicios web modernos (Google, Twitter, GitHub, Spotify) lo usan.

Desventajas de JSON

1. Tipos de datos limitados

JSON solo maneja 6 tipos básicos: string, number, boolean, null, array y object.

No existen tipos como:

•Fechas — deben guardarse como strings ("2025-09-09") o números (timestamp).

•Binarios — suelen codificarse en Base64, lo que aumenta el tamaño.

2. No permite comentarios

JSON no admite comentarios (`//` o `/* */`). Esto dificulta documentar configuraciones directamente en el archivo.

Ejemplo inválido:

```
{
  "usuario": "Ana" // Este es el nombre del usuario
}
```

3. Redundancia en datos muy grandes

Al usar siempre claves como strings, archivos JSON grandes pueden ser más pesados que formatos binarios especializados.

Ejemplo: En una base con millones de registros, repetir "nombre", "edad", "activo" en cada objeto ocupa mucho espacio.

4. Menos seguro si no se valida

Como JSON se interpreta en muchos lenguajes, si no se valida puede ser vulnerable a ataques (ejemplo: inyección de JSON malicioso).
Por eso se recomienda siempre usar validación con JSON Schema o librerías oficiales.

5. No ideal para configuraciones muy complejas

En algunos casos, YAML o TOML son preferidos para configuraciones (ejemplo: Docker Compose, Kubernetes), ya que permiten comentarios y una sintaxis más expresiva.

Comparación rápida con otros formatos

Característica	JSON	XML	YAML
Legible por humanos	Alta	Media	Alta
Tamaño de archivo	Pequeño	Grande	Medio
Comentarios	No	Sí	Sí
Tipos de datos	Limitados	Flexibles	Flexibles
Estándar web	Sí	Antes lo fue	No es estándar oficial
Complejidad	Baja	Alta	Media

Resumen

Ventajas: Simple, ligero, universal, ideal para APIs, ampliamente soportado.
Desventajas: Tipos de datos limitados, sin comentarios, redundante en grandes volúmenes, menos seguro si no se valida.
👉 En conclusión, JSON es el formato más usado hoy en la web y las APIs gracias a su simplicidad y eficiencia, aunque en casos específicos (configuraciones complejas, datos binarios, validaciones estrictas) se prefieren otros formatos.

JSON vs otros formatos (XML, YAML, CSV)

JSON no es el único formato de intercambio de datos. Existen otros que cumplen funciones similares o complementarias, como XML, YAML y CSV. Cada uno tiene ventajas, desventajas y escenarios de uso específicos.

📌 JSON (JavaScript Object Notation)

•Sintaxis: basada en objetos `{}` y arreglos `[]`.
•Tipos soportados: string, number, boolean, null, array, object.
•Usos típicos: APIs REST, aplicaciones web/móviles, bases de datos NoSQL (MongoDB).

Ejemplo:

```
{
  "usuario": "Ana",
  "edad": 28,
  "activo": true
}
```

Ventajas: simple, ligero, universal, muy usado en la web.

Desventajas: no permite comentarios, tipos de datos limitados.

📌 XML (eXtensible Markup Language)

•Sintaxis: basada en etiquetas de apertura `< >` y cierre `</ >`.

•Tipos soportados: más flexible que JSON (atributos, namespaces, validación con XSD).

•Usos típicos: documentos estructurados, sistemas legados, SOAP, configuración en algunos entornos.

Ejemplo:

```
<usuario activo="true">
  <nombre>Ana</nombre>
  <edad>28</edad>
</usuario>
```

Ventajas:

•Permite comentarios.

•Muy flexible (puede modelar datos complejos).

•Estándar consolidado en sistemas empresariales.

Desventajas:

•Muy verboso (consume más espacio y ancho de banda).

•Más difícil de leer para humanos.

•Procesamiento más lento en comparación con JSON.

📌 YAML (YAML Ain't Markup Language)

•Sintaxis: basada en indentación (espacios) en lugar de corchetes o llaves.

•Tipos soportados: admite más expresividad que JSON (fechas, referencias, objetos anidados sin tanta sintaxis).

•Usos típicos: archivos de configuración (Docker, Kubernetes, GitHub Actions).

Ejemplo:

```
usuario: Ana
edad: 28
activo: true
hobbies:
  - leer
  - correr
```

```
  - viajar
```

Ventajas:

•Muy legible para humanos (sin comillas ni corchetes).

•Admite comentarios con #.

•Ideal para configuraciones extensas.

Desventajas:

•Más propenso a errores por indentación incorrecta.

•No es tan estándar en web como JSON.

•Parseadores menos universales en algunos lenguajes.

📌 CSV (Comma-Separated Values)

•Sintaxis: tabla en texto plano, separada por comas (o punto y coma).

•Tipos soportados: solo strings y números, sin soporte para estructuras complejas.

•Usos típicos: intercambio de datos tabulares (Excel, bases de datos, Google Sheets).

Ejemplo:

```
usuario,edad,activo
Ana,28,true
Luis,35,false
```

Ventajas:

•Muy ligero.

•Fácil de abrir en Excel o Google Sheets.

•Ideal para datos tabulares simples.

Desventajas:

•No soporta jerarquías ni estructuras anidadas.

•No maneja tipos más complejos (boolean, null, objetos).

•Ambigüedad con comas y saltos de línea en valores.

📊 Comparación directa

Característica	JSON	XML	YAML	CSV
Legibilidad humana	Alta	Media (verboso)	Muy alta	Alta (datos simples)
Tamaño de archivo	Pequeño	Grande	Medio	Muy pequeño
Comentarios	No	Sí	Sí	No
Estructuras anidadas	Sí	Sí	Sí	No
Tipos de datos	Limitados (6 básicos)	Flexibles	Flexibles (fechas, referencias)	Muy limitados
Popularidad en la web	Muy alta	Baja (antes muy usado)	Alta (configuraciones)	Alta (datos tabulares)
Facilidad de parseo	Muy fácil	Complejo	Medio	Muy fácil

¿Cuándo usar cada uno?

•JSON: cuando necesites transmitir datos entre cliente y servidor, APIs REST, aplicaciones móviles o guardar configuraciones sencillas.
•XML: en sistemas legados, documentos que requieran validación estricta, o cuando se usen estándares antiguos como SOAP.
•YAML: en configuraciones complejas que necesiten ser fáciles de leer y modificar por humanos (ejemplo: DevOps).
•CSV: en exportación e importación de datos tabulares simples (Excel, bases de datos, informes).

Resumen

•JSON es el formato universal actual para web y APIs.
•XML aún sobrevive en sistemas antiguos y documentos complejos.
•YAML domina en el mundo de configuración de infraestructuras.
•CSV sigue siendo el rey para datos tabulares simples.
👉 Ningún formato es mejor en todo; cada uno se adapta mejor a un contexto específico.

Lectura y escritura de JSON en JavaScript

JavaScript es el lenguaje nativo de JSON, ya que el formato fue creado inspirado en la notación de objetos de este lenguaje. Por eso, trabajar con JSON en JavaScript es muy sencillo y directo.

¿Cómo maneja JSON JavaScript?

JavaScript provee dos funciones globales para trabajar con JSON:
•JSON.parse() — Convierte un string en formato JSON a un objeto de JavaScript.
•JSON.stringify() — Convierte un objeto de JavaScript a un string en formato JSON.

1. Convertir un JSON a objeto con JSON.parse()

Cuando recibimos datos en formato JSON (por ejemplo, desde una API o archivo externo), vienen como string. Para poder manipularlos en JavaScript, hay que convertirlos a objeto.
Ejemplo:

```
const jsonString = '{"nombre": "Ana", "edad": 28, "activo":
true}';

// Convertimos el string JSON a objeto de JavaScript
const objeto = JSON.parse(jsonString);
```

```
console.log(objeto.nombre);  // Ana
console.log(objeto.edad);    // 28
console.log(objeto.activo);  // true
```

🔎 Observación: Después de JSON.parse(), podemos acceder a los datos como propiedades normales de un objeto.

2. Convertir un objeto a JSON con JSON.stringify()

Cuando queremos enviar datos a un servidor o guardarlos, necesitamos pasarlos a formato JSON.

Ejemplo:

```
const usuario = {
  nombre: "Luis",
  edad: 35,
  activo: false
};

// Convertimos el objeto a string JSON
const jsonString = JSON.stringify(usuario);

console.log(jsonString);
// {"nombre":"Luis","edad":35,"activo":false}
```

🔎 Observación: El resultado es un string válido en formato JSON. Este string es lo que viaja en las peticiones HTTP (por ejemplo, en fetch).

3. Ejemplo práctico: Enviar JSON con fetch

En aplicaciones web, lo más común es enviar y recibir JSON de un servidor.

Ejemplo de enviar datos:

```
const nuevoProducto = {
  codigo: 101,
  descripcion: "Teclado mecánico",
  precio: 1200.50
};

fetch("https://miapi.com/productos", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(nuevoProducto)  // enviamos JSON al servidor
})
  .then(respuesta => respuesta.json()) // interpretamos la
respuesta como JSON
  .then(data => console.log("Servidor respondió:", data))
  .catch(error => console.error("Error:", error));
```

Ejemplo de recibir datos:

```
fetch("https://miapi.com/productos")
  .then(respuesta => respuesta.json()) // parseamos la respuesta
JSON
  .then(data => {
    console.log("Productos disponibles:");
    console.log(data);
  })
  .catch(error => console.error("Error:", error));
```

4. Ejemplo práctico en navegador (localStorage)

Podemos usar JSON para guardar información del usuario en el navegador.

Ejemplo:

```
// Guardar un objeto en localStorage
const usuario = { nombre: "María", edad: 29 };
localStorage.setItem("usuario", JSON.stringify(usuario));

// Recuperar y parsear
const datosGuardados = localStorage.getItem("usuario");
const usuarioRecuperado = JSON.parse(datosGuardados);

console.log(usuarioRecuperado.nombre); // María
```

Referencia: localStorage.

5. Manejo de errores en JSON.parse()

Si el string JSON tiene un error de sintaxis, JSON.parse() lanza una excepción.

Ejemplo:

```
const jsonInvalido = "{nombre: 'Juan'}"; // incorrecto (faltan
comillas dobles)

try {
  const obj = JSON.parse(jsonInvalido);
} catch (error) {
  console.error("Error al parsear JSON:", error.message);
}
```

6. Formateo y legibilidad con JSON.stringify()

JSON.stringify() admite parámetros extra para formatear con sangría y mejorar la legibilidad.

Ejemplo:

```
const datos = { nombre: "Carlos", edad: 40 };

const jsonFormateado = JSON.stringify(datos, null, 2);
console.log(jsonFormateado);
// Salida:
// {
//   "nombre": "Carlos",
//   "edad": 40
```

```
// }
```

📊 Resumen

•JSON.parse() — Convierte string JSON a objeto JavaScript.
•JSON.stringify() — Convierte objeto JavaScript a string JSON.
•JSON es el formato estándar de comunicación en fetch (cliente-servidor).
•Se puede usar también en almacenamiento local (localStorage).
•Es importante manejar errores al parsear para evitar fallas en la app.

Lectura y escritura de JSON en Python (módulo json)

Python tiene soporte nativo para JSON a través del módulo estándar json. Esto significa que no hace falta instalar nada extra: viene incluido con Python.
Este módulo permite:
•Convertir un string JSON a un diccionario de Python.
•Convertir un diccionario de Python a un string JSON.
•Leer y escribir archivos en formato JSON.

1. Importar el módulo json

```
import json
```

2. Convertir JSON a diccionario (json.loads)

Cuando recibimos datos en formato JSON (por ejemplo, desde una API), vienen como string. Podemos convertir ese string a un diccionario de Python usando json.loads().
Ejemplo:

```
import json

json_string = '{"nombre": "Ana", "edad": 28, "activo": true}'

# Convertir a diccionario de Python
objeto = json.loads(json_string)

print(objeto["nombre"])  # Ana
print(objeto["edad"])    # 28
print(objeto["activo"])  # True
```

Observaciones:
•En JSON — true/false; en Python — True/False.
•En JSON — null; en Python — None.

3. Convertir diccionario a JSON (json.dumps)

Si queremos enviar datos a un servidor o guardarlos en un archivo, debemos convertir un diccionario a JSON con json.dumps().

Ejemplo:

```
import json

usuario = {
    "nombre": "Luis",
    "edad": 35,
    "activo": False
}

# Convertir a JSON (string)
json_string = json.dumps(usuario)

print(json_string)
# {"nombre": "Luis", "edad": 35, "activo": false}
```

4. Formatear JSON con indent y sort_keys

Podemos hacer que el JSON generado sea más legible con sangría (indent) y orden de claves (sort_keys).

Ejemplo:

```
import json

datos = {"nombre": "Carlos", "edad": 40, "activo": True}

json_formateado = json.dumps(datos, indent=4, sort_keys=True)
print(json_formateado)

# Salida:
# {
#     "activo": true,
#     "edad": 40,
#     "nombre": "Carlos"
# }
```

5. Leer JSON desde un archivo (json.load)

Además de trabajar con strings, Python puede leer archivos JSON directamente.

Ejemplo (archivo usuario.json):

```
{
  "nombre": "María",
  "edad": 29,
  "activo": true
}
```

Código en Python:

```
import json
```

```
with open("usuario.json", "r", encoding="utf-8") as archivo:
    datos = json.load(archivo)

print(datos["nombre"])  # María
```

6. Escribir JSON en un archivo (json.dump)

Podemos guardar un diccionario de Python en un archivo .json.

Ejemplo:

```
import json

producto = {
    "codigo": 101,
    "descripcion": "Teclado mecánico",
    "precio": 1200.50,
    "disponible": True
}

with open("producto.json", "w", encoding="utf-8") as archivo:
    json.dump(producto, archivo, indent=4)
```

Esto genera un archivo producto.json con contenido como:

```
{
    "codigo": 101,
    "descripcion": "Teclado mecánico",
    "precio": 1200.5,
    "disponible": true
}
```

7. Manejo de errores al parsear JSON

Si el JSON está mal escrito, Python lanza un json.JSONDecodeError.

Ejemplo:

```
import json

json_invalido = '{nombre: "Ana"}'  # Incorrecto: faltan comillas
dobles en la clave

try:
    datos = json.loads(json_invalido)
except json.JSONDecodeError as e:
    print("Error al parsear JSON:", e)
```

8. Conversión entre tipos JSON ↔️ Python

JSON ↔️ Python

object ↔️ dict

array ↔️ list

string ↔️ str

number ↔️ int/float

true ↔️ True

false ↔️ False

null ↔️ None

Ejemplo:

```
import json

json_string = '{"numero": 10, "activo": false, "extra": null}'
objeto = json.loads(json_string)

print(type(objeto["numero"]))  # <class 'int'>
print(type(objeto["activo"]))  # <class 'bool'>
print(type(objeto["extra"]))   # <class 'NoneType'>
```

📊 Resumen

•json.loads() — Convierte string JSON a diccionario Python.

•json.dumps() — Convierte diccionario Python a string JSON.

•json.load() — Lee JSON desde un archivo.

•json.dump() — Escribe JSON a un archivo.

•Admite formateo (indent, sort_keys).

•Errores de sintaxis generan JSONDecodeError.

Lectura y escritura de JSON en Java (biblioteca Jackson / Gson)

En Java, a diferencia de Python o JavaScript, JSON no tiene soporte nativo directo. Por eso se utilizan bibliotecas externas muy populares como Jackson y Gson (de Google). Ambas permiten serializar (convertir objetos Java a JSON) y deserializar (convertir JSON a objetos Java).

Opción 1: Usando Jackson

Jackson es una de las bibliotecas más usadas en Java para trabajar con JSON.

1) Dependencia (Maven)

Agrega en tu Maven pom.xml:

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.15.2</version>
```

```
</dependency>
```

2) Crear una clase Java (POJO)

```
public class Usuario {
    private String nombre;
    private int edad;
    private boolean activo;

    // Constructor vacío requerido por Jackson
    public Usuario() {}

    // Constructor completo
    public Usuario(String nombre, int edad, boolean activo) {
        this.nombre = nombre;
        this.edad = edad;
        this.activo = activo;
    }

    // Getters y Setters
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }

    public boolean isActivo() { return activo; }
    public void setActivo(boolean activo) { this.activo = activo; }
}
```

3) Serializar objeto → JSON

```
import com.fasterxml.jackson.databind.ObjectMapper;

public class Main {
    public static void main(String[] args) throws Exception {
        Usuario usuario = new Usuario("Ana", 28, true);

        ObjectMapper mapper = new ObjectMapper();
        String jsonString = mapper.writeValueAsString(usuario);

        System.out.println(jsonString);
        // {"nombre":"Ana","edad":28,"activo":true}
    }
}
```

4) Deserializar JSON → objeto

```
import com.fasterxml.jackson.databind.ObjectMapper;

public class Main {
    public static void main(String[] args) throws Exception {
```

```
        String jsonString =
"{\"nombre\":\"Luis\",\"edad\":35,\"activo\":false}";

        ObjectMapper mapper = new ObjectMapper();
        Usuario usuario = mapper.readValue(jsonString,
Usuario.class);

        System.out.println(usuario.getNombre()); // Luis
        System.out.println(usuario.getEdad());   // 35
        System.out.println(usuario.isActivo());  // false
    }
}
```

5) Leer y escribir JSON en archivo

```
import java.io.File;
import com.fasterxml.jackson.databind.ObjectMapper;

public class Main {
    public static void main(String[] args) throws Exception {
        Usuario usuario = new Usuario("Carlos", 40, true);
        ObjectMapper mapper = new ObjectMapper();

        // Escribir en archivo
        mapper.writeValue(new File("usuario.json"), usuario);

        // Leer desde archivo
        Usuario cargado = mapper.readValue(new File("usuario.json"),
Usuario.class);
        System.out.println("Usuario leído: " + cargado.getNombre());
    }
}
```

Opción 2: Usando Gson (Google)

Gson es otra biblioteca muy usada, ligera y sencilla.

1) Dependencia (Maven)

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
</dependency>
```

2) Serializar objeto → JSON

```
import com.google.gson.Gson;

public class Main {
    public static void main(String[] args) {
        Usuario usuario = new Usuario("María", 30, true);
```

```
        Gson gson = new Gson();
        String jsonString = gson.toJson(usuario);

        System.out.println(jsonString);
        // {"nombre":"María","edad":30,"activo":true}
    }
}
```

3) Deserializar JSON → objeto

```
import com.google.gson.Gson;

public class Main {
    public static void main(String[] args) {
        String jsonString =
"{\"nombre\":\"Pedro\",\"edad\":25,\"activo\":false}";

        Gson gson = new Gson();
        Usuario usuario = gson.fromJson(jsonString, Usuario.class);

        System.out.println(usuario.getNombre()); // Pedro
        System.out.println(usuario.getEdad());   // 25
    }
}
```

Comparación Jackson vs Gson

Característica	Jackson	Gson
Popularidad	Muy alta (Spring Boot lo usa)	Alta (creada por Google)
Facilidad	Más completa pero más pesada	Ligera y sencilla
Performance	Muy rápida en grandes datasets	Adecuada, pero más lenta en casos masivos
Extras	Buen ecosistema; soporte de herramientas como JSON Schema, integración con YAML vía módulos	Menos funcionalidades extra

Resumen

•En Java, JSON se maneja con bibliotecas externas.

•Jackson — robusta, estándar en proyectos grandes.

•Gson — ligera, fácil de usar.

•Ambas permiten serializar (Java → JSON) y deserializar (JSON → Java) de forma simple.

Uso de JSON en APIs REST (request y response en formato JSON)

1) Concepto básico

REST es un estilo de arquitectura para servicios web.
JSON es el formato de datos más usado en REST por su ligereza y compatibilidad.
La app cliente (web/móvil/otro servicio) envía HTTP requests y la API responde con HTTP responses (status code + headers + cuerpo JSON).

2) Estructura de un request típico (HTTP)

Componentes clave:
•Método: GET, POST, PUT, PATCH, DELETE.
•URL (recurso): https://api.ejemplo.com/productos/101
•Headers:
Content-Type: application/json (si envías body)
Accept: application/json (lo que esperas recibir)
Authorization: Bearer <token> (si hay auth)
•Body (en métodos con cuerpo): JSON válido
Ejemplo POST (crear)

```
POST /productos HTTP/1.1
Host: api.ejemplo.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer <token>

{
  "descripcion": "Teclado mecánico",
  "precio": 1200.50,
  "disponible": true
}
```

3) Estructura de un response típico (HTTP)

•Status code (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error)
•Headers (p. ej. Content-Type: application/json; charset=utf-8)
•Body — JSON con datos o error
Ejemplo 201 Created

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
Location: https://api.ejemplo.com/productos/101

{
  "id": 101,
  "descripcion": "Teclado mecánico",
  "precio": 1200.50,
  "disponible": true
}
```

4) Métodos y semántica

•GET: leer (no tiene body; seguro).

•POST: crear .

•PUT: reemplazar recurso completo.

•PATCH: actualizar parcialmente.

•DELETE: borrar recurso.

5) Ejemplos de consumo (cliente)

a) curl (CLI)

```
# GET (listar)
curl -H "Accept: application/json"
https://api.ejemplo.com/productos

# POST (crear)
curl -X POST https://api.ejemplo.com/productos \
  -H "Content-Type: application/json" -H "Accept: application/json"
\
  -d '{"descripcion":"Teclado","precio":1200.5,"disponible":true}'
```

Referencia: curl.

b) JavaScript (fetch)

```
// GET
const lista = await fetch("/productos", { headers: { Accept:
"application/json" } })
  .then(r => r.json());

// POST
const nuevo = await fetch("/productos", {
  method: "POST",
  headers: { "Content-Type": "application/json", "Accept":
"application/json" },
  body: JSON.stringify({ descripcion: "Mouse", precio: 850.75,
disponible: false })
}).then(r => r.json());
```

Docs: fetch.

c) Python (requests)

```
import requests
r = requests.get("https://api.ejemplo.com/productos",
headers={"Accept": "application/json"})
productos = r.json()

nuevo = {"descripcion": "Monitor 24", "precio": 1500.0,
"disponible": True}
r = requests.post("https://api.ejemplo.com/productos",
                  json=nuevo,
                  headers={"Accept":"application/json"})
data = r.json()
```

Docs: requests.

d) C# (.NET HttpClient)

```
using System.Net.Http.Json;

var http = new HttpClient { BaseAddress = new
Uri("https://api.ejemplo.com/") };
var productos = await
http.GetFromJsonAsync<List<Producto>>("productos");

var creado = await http.PostAsJsonAsync("productos", new
{ descripcion="Parlantes", precio=999.99, disponible=true });
var body = await creado.Content.ReadFromJsonAsync<Producto>();
```

Docs: System.Net.Http.Json.

6) Diseño de respuestas JSON

a) Recurso simple

```
{
  "id": 101,
  "descripcion": "Teclado mecánico",
  "precio": 1200.5,
  "disponible": true
}
```

b) Colección con paginación y filtros

Query params comunes: `?page=2&limit=20&sort=precio:desc&search=teclado`

```
{
  "data": [
    { "id": 101, "descripcion": "Teclado", "precio": 1200.5,
"disponible": true },
    { "id": 145, "descripcion": "Teclado RGB", "precio": 1800.0,
"disponible": true }
  ],
```

```
  "pagination": { "page": 2, "limit": 20, "total": 87, "pages":
5 },
  "filters": { "search": "teclado", "sort": "precio:desc" }
}
```

Plano: devolvés la lista directo (más simple). Envoltorio: data + pagination + meta (más extensible). Elegí uno y sé consistente.

7) Errores en JSON (formato recomendado)

Usá un formato consistente y legible. Referencia útil: RFC 7807 (application/problem+json).

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://api.ejemplo.com/errors/validation",
  "title": "Error de validación",
  "status": 400,
  "detail": "El campo 'precio' debe ser mayor a 0.",
  "errors": {
    "precio": ["Debe ser mayor a 0"]
  }
}
```

8) Autenticación y autorización (headers)

•Bearer Token (JWT o similar): Authorization: Bearer <token>

•API Keys: x-api-key: <clave>

•OAuth2/OpenID Connect para escenarios avanzados.

Respuestas ante problemas: 401 Unauthorized (no autenticado) y 403 Forbidden (autenticado pero sin permisos).

9) CORS (navegadores)

Permite que un frontend en dominio A consuma API en dominio B. Ver CORS.

Respuestas de la API deben incluir headers como:

```
Access-Control-Allow-Origin: https://mi-frontend.com
Access-Control-Allow-Methods: GET,POST,PUT,DELETE,OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization
```

10) Versionado de API

•En la URL: /v1/productos

•En header (content negotiation): Accept: application/vnd.miapi.v2+json

11) Caching (mejor rendimiento)

Headers: Cache-Control, ETag, Last-Modified. Ver caché HTTP.

Flujo típico:

```
Server responde 200 OK + ETag: "abc123".
Cliente reintenta con If-None-Match: "abc123".
Si no cambió: 304 Not Modified (sin body).
```

12) Idempotencia y seguridad

•GET: seguro e idempotente (no cambia estado).

•PUT/DELETE: idempotentes (múltiples llamadas = mismo resultado).

•POST: no idempotente (cuidado con reintentos; usar claves de idempotencia si la plataforma lo permite).

13) PATCH (actualización parcial)

Dos estilos comunes:

•Merge Patch (application/merge-patch+json): enviás solo campos a cambiar.

Especificación: RFC 7396.

•JSON Patch (application/json-patch+json): operaciones (add, replace, remove).

Especificación: RFC 6902.

Ejemplo Merge Patch

```
PATCH /productos/101
Content-Type: application/merge-patch+json

{ "precio": 1399.99 }
```

14) Validación con JSON Schema (servidor/cliente)

Define reglas de tipos, requeridos y rangos. Útil para contratos y tests (contract testing). Ver JSON Schema.

15) Estándares de formato de respuesta

•OpenAPI/Swagger: documentar endpoints, modelos y ejemplos JSON.

•JSON:API / HAL / Problem+JSON: convenciones para enlaces, errores, paginación, etc.

16) Seguridad y robustez

•Validá y saneá todo input JSON (tipos, rangos, longitud).

•Limitá tamaño de request (max body size) para evitar abusos.

•Rate limiting / throttling.

•Logs sin datos sensibles (o enmascarados).

•HTTPS siempre.

17) Mini API de ejemplo (contrato)

Recursos

```
GET /productos         lista (paginada)
GET /productos/{id}  detalle
POST /productos      crear
PUT /productos/{id}  reemplazar
```

```
PATCH /productos/{id} actualizar parcial
DELETE /productos/{id} borrar
```

Modelo (JSON)

```
{
  "id": 101,
  "descripcion": "Teclado mecánico",
  "precio": 1200.5,
  "disponible": true,
  "tags": ["periférico", "oferta"]
}
```

Respuesta paginada (GET /productos?page=1&limit=2)

```
{
  "data": [
    { "id": 101, "descripcion": "Teclado", "precio": 1200.5,
"disponible": true },
    { "id": 102, "descripcion": "Mouse", "precio": 850.75,
"disponible": false }
  ],
  "pagination": { "page": 1, "limit": 2, "total": 37, "pages": 19 }
}
```

Error de validación (POST)

```
{
  "type": "https://api.ejemplo.com/errors/validation",
  "title": "Error de validación",
  "status": 400,
  "errors": {
    "descripcion": ["Debe tener al menos 3 caracteres"],
    "precio": ["Debe ser mayor a 0"]
  }
}
```

18) Checklist de buenas prácticas

•Content-Type y Accept en application/json.

•Respuestas coherentes (mismo formato, mismas claves).

•Status codes correctos (200/201/204/400/401/403/404/409/422/500).

•Paginación y filtros en listados.

•Errores estructurados (RFC 7807).

•Documentación con OpenAPI.

•Auth clara (Bearer JWT, API Key, OAuth2).

•CORS configurado.

•Caching con ETag/Cache-Control cuando corresponda.

•Versionado (URL o header).

•Validación con JSON Schema.

Resumen

JSON es el formato rey de las APIs REST. Dominar headers, métodos, status codes y estructuras de respuesta te permite diseñar y consumir APIs sólidas. Sumá paginación, errores consistentes, validación y seguridad para un API profesional.

JSON Schema: validación y definición de datos

1) ¿Qué es JSON Schema?

JSON Schema es un estándar para describir, validar y documentar la estructura de un documento JSON.

Permite definir qué campos existen, qué tipo de datos tienen y qué restricciones deben cumplir. Es equivalente a un "contrato" entre quien envía y quien recibe el JSON.

Sirve para validar automáticamente datos en APIs REST, formularios web o bases de datos NoSQL.

2) Ejemplo simple

JSON a validar

```
{
  "nombre": "Ana",
  "edad": 28,
  "activo": true
}
```

JSON Schema correspondiente

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "nombre": { "type": "string" },
    "edad": { "type": "integer", "minimum": 0 },
    "activo": { "type": "boolean" }
  },
  "required": ["nombre", "edad"]
}
```

Significado:

El objeto debe tener propiedades nombre, edad, activo.

nombre debe ser string; edad entero ≥ 0; activo booleano.

nombre y edad son obligatorios.

3) Elementos básicos de JSON Schema

•type — Define el tipo (string, number, integer, boolean, null, object, array).

•properties — Define las claves de un objeto y sus restricciones.

•required — Lista de campos obligatorios.

•items — Estructura de los elementos de un array.

•minimum, maximum, minLength, maxLength — Restricciones numéricas y de texto.

•enum — Lista de valores permitidos.

•pattern — Validación con expresiones regulares para strings.

4) Ejemplo con validaciones

JSON a validar

```
{
  "email": "user@example.com",
  "rol": "admin",
  "edad": 25
}
```

JSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "email": { "type": "string", "format": "email" },
    "rol": { "type": "string", "enum": ["admin", "editor",
"usuario"] },
    "edad": { "type": "integer", "minimum": 18, "maximum": 99 }
  },
  "required": ["email", "rol"]
}
```

Validaciones:

email debe tener formato válido.

rol debe ser uno de los valores permitidos.

edad debe estar entre 18 y 99.

5) Validar arrays

JSON a validar

```
{
  "tags": ["tecnología", "json", "api"]
}
```

JSON Schema

```
{
  "type": "object",
  "properties": {
    "tags": {
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1,
      "uniqueItems": true
    }
  }
}
```

6) Objetos anidados

JSON a validar

```
{
  "usuario": {
    "nombre": "María",
    "direccion": {
      "calle": "San Martín",
      "numero": 123,
      "ciudad": "Córdoba"
    }
  }
}
```

JSON Schema

```
{
  "type": "object",
  "properties": {
    "usuario": {
      "type": "object",
      "properties": {
        "nombre": { "type": "string" },
        "direccion": {
          "type": "object",
          "properties": {
            "calle": { "type": "string" },
            "numero": { "type": "integer" },
            "ciudad": { "type": "string" }
          },
          "required": ["calle", "numero"]
        }
      },
      "required": ["nombre", "direccion"]
    }
  }
}
```

7) Ejemplo avanzado: catálogo de productos

JSON válido

```
{
  "tienda": "Tech Store",
  "productos": [
    {
      "codigo": 101,
      "descripcion": "Teclado",
      "precio": 1200.50,
      "disponible": true,
      "categorias": ["periféricos", "ofertas"]
    }
  ]
}
```

JSON Schema

```
{
  "type": "object",
  "properties": {
    "tienda": { "type": "string" },
    "productos": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "codigo": { "type": "integer" },
          "descripcion": { "type": "string", "minLength": 3 },
          "precio": { "type": "number", "minimum": 0 },
          "disponible": { "type": "boolean" },
          "categorias": {
            "type": "array",
            "items": { "type": "string" }
          }
        },
        "required": ["codigo", "descripcion", "precio"]
      }
    }
  },
  "required": ["tienda", "productos"]
}
```

8) Validación en distintos lenguajes

Python (con jsonschema)

```
from jsonschema import validate, ValidationError

schema = {
    "type": "object",
```

```
    "properties": {
        "nombre": {"type": "string"},
        "edad": {"type": "integer", "minimum": 0}
    },
    "required": ["nombre", "edad"]
}

data = {"nombre": "Ana", "edad": 25}

try:
    validate(instance=data, schema=schema)
    print("JSON válido")
except ValidationError as e:
    print("Error:", e.message)
```

Lib: jsonschema.

JavaScript (con ajv)

```
import Ajv from "ajv";
const ajv = new Ajv();

const schema = {
  type: "object",
  properties: {
    nombre: { type: "string" },
    edad: { type: "integer", minimum: 0 }
  },
  required: ["nombre", "edad"]
};

const validate = ajv.compile(schema);
const valid = validate({ nombre: "Ana", edad: 25 });

console.log(valid ? "JSON válido" : "Error", validate.errors);
```

Lib: AJV.

9) Usos prácticos de JSON Schema

•Validación de entradas en APIs REST — asegurar que el cliente envía datos correctos.

•Documentación — describe qué estructura se espera en cada endpoint.

•Generación automática de formularios (ej.: React JSON Schema Form).

•Contratos entre microservicios — asegura que todos usan el mismo formato.

•Testing — validar que las respuestas de una API cumplen con lo especificado.

Resumen

•JSON Schema define reglas sobre qué estructura debe tener un JSON.

•Permite especificar tipos, obligatorios, rangos, enums, arrays y objetos anidados.

•Existen librerías en casi todos los lenguajes (Python: jsonschema, JS: AJV, Java: Everit, Go: gojsonschema).
•Es fundamental para APIs modernas, microservicios y validación de datos.

Uso de JSON en bases de datos NoSQL (ejemplo: MongoDB)

1) Introducción

Las bases de datos NoSQL (Not Only SQL) surgieron para manejar grandes volúmenes de datos flexibles y semiestructurados. Una de las más populares es MongoDB, que utiliza un formato muy cercano a JSON llamado BSON (Binary JSON).
•JSON — formato de texto para representar datos.
•BSON — versión binaria de JSON usada internamente en MongoDB, que añade tipos extra (fechas, binarios, ObjectId).
En la práctica, cuando trabajamos con MongoDB usamos documentos en JSON para insertar, consultar y actualizar.

2) Documentos JSON en MongoDB

En lugar de tablas y filas (SQL), MongoDB usa:
•Colecciones (equivalentes a tablas).
•Documentos (equivalentes a filas).
Ejemplo de documento (usuario):

```
{
  "_id": "ObjectId(64f8c7e12345abc67890def1)",
  "nombre": "Ana",
  "edad": 28,
  "email": "ana@example.com",
  "activo": true,
  "direccion": {
    "calle": "San Martín",
    "numero": 123,
    "ciudad": "Córdoba"
  },
  "hobbies": ["leer", "correr", "viajar"]
}
```

3) Inserción de documentos JSON

En la consola de MongoDB (mongosh):

```
db.usuarios.insertOne({
  nombre: "Luis",
```

```
  edad: 35,
  email: "luis@example.com",
  activo: false
});

// Insertar varios
db.usuarios.insertMany([
  { nombre: "Carlos", edad: 40, activo: true },
  { nombre: "María", edad: 29, activo: true }
]);
```

4) Consultas con JSON

```
// Consultar todos
db.usuarios.find();

// Consultar con condición
db.usuarios.find({ activo: true });

// Proyección (solo ciertos campos)
db.usuarios.find(
  { activo: true },
  { nombre: 1, email: 1, _id: 0 }
);
// Resultado ejemplo: { "nombre": "María", "email":
"maria@example.com" }
```

5) Actualización con JSON

```
// Actualizar un campo
db.usuarios.updateOne(
  { nombre: "Luis" },
  { $set: { activo: true } }
);

// Actualizar varios documentos
db.usuarios.updateMany(
  { activo: false },
  { $set: { activo: true } }
);
```

6) Eliminación de documentos

```
// Eliminar uno
db.usuarios.deleteOne({ nombre: "Carlos" });

// Eliminar varios
db.usuarios.deleteMany({ activo: false });
```

7) Ejemplo de estructura compleja

En MongoDB, es común usar documentos JSON anidados:

```
{
  "_id": "ObjectId(64f8c7e12345abc67890def2)",
```

```
  "pedido": 5001,
  "cliente": {
    "nombre": "María",
    "email": "maria@example.com"
  },
  "items": [
    { "producto": "Notebook", "cantidad": 1, "precio": 2500.00 },
    { "producto": "Mouse", "cantidad": 2, "precio": 850.75 }
  ],
  "total": 4201.50,
  "estado": "pendiente"
}
```

En una base SQL tradicional, estos datos estarían normalizados en varias tablas. En MongoDB, todo se guarda en un solo documento JSON.

8) Validación de JSON en MongoDB

MongoDB permite definir schemas de validación (similar a JSON Schema).

Ejemplo: solo aceptar documentos de usuarios con nombre y edad obligatorios.

```
db.createCollection("usuarios", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "edad"],
      properties: {
        nombre: { bsonType: "string" },
        edad: { bsonType: "int", minimum: 0 }
      }
    }
  }
});

// Intento inválido (falta "edad")
db.usuarios.insertOne({ nombre: "Pedro" }); // Error de validación
```

9) Conexión desde lenguajes de programación

Python (PyMongo)

```
from pymongo import MongoClient

cliente = MongoClient("mongodb://localhost:27017/")
db = cliente["mi_base"]

usuario = {"nombre": "Ana", "edad": 28, "activo": True}
db.usuarios.insert_one(usuario)

for u in db.usuarios.find({"activo": True}):
    print(u)
```

Lib: PyMongo.

Node.js (Driver oficial)

```
import { MongoClient } from "mongodb";

async function run() {
  const client = new MongoClient("mongodb://localhost:27017");
  await client.connect();
  const db = client.db("mi_base");

  await db.collection("usuarios").insertOne({ nombre: "Luis", edad:
30, activo: true });

  const usuarios = await db.collection("usuarios").find({ activo:
true }).toArray();
  console.log(usuarios);

  await client.close();
}
run();
```

Lib: MongoDB Node.js Driver.

10) Ventajas de usar JSON en MongoDB

•Flexibilidad — No es necesario definir un esquema fijo como en SQL.

•Natural para aplicaciones — APIs REST trabajan en JSON, y MongoDB también.

•Documentos anidados — Menos necesidad de joins.

•Ecosistema — Muy usado en proyectos de Big Data y microservicios.

Consideraciones:

•Validar bien los datos; la flexibilidad puede llevar a inconsistencias.

•Documentos muy grandes pueden complicar consultas; balancear entre anidación y normalización.

Resumen

•MongoDB almacena datos en formato BSON (extensión binaria de JSON).

•Se trabaja con colecciones (tablas) y documentos JSON (filas).

•Operaciones CRUD (insert, find, update, delete) se realizan con JSON.

•Permite anidación de objetos y arrays de forma natural.

•Se pueden definir schemas de validación similares a JSON Schema.

•Integración fácil con lenguajes modernos (Python, Node.js, Java, etc.).

Buenas prácticas al trabajar con JSON

1) Nombres de claves

•Consistencia: elegí camelCase o snake_case y mantenelo.
•Claridad > abreviaturas: precioUnitario mejor que prcU.
•Unidades explícitas cuando aplique: pesoKg, duracionMs, montoArs.
•Plural vs singular: listas en plural (items), elemento en singular (item).

2) Tipos de datos correctos

•Boolean reales (true/false), nunca strings "true".
•Números: preferí enteros para contadores; para dinero, usá decimal (o centavos como int).
•Fechas/horas: ISO-8601 en UTC, p. ej. "2025-09-09T16:30:00Z"; con zona, incluí offset.
•Binarios: codificá en Base64.
•Enum: restringí con listas permitidas ("estado": "pendiente"|"pagado"|"cancelado").

3) null vs campo ausente

•Campo ausente: no aplica / desconocido.
•null: aplica pero sin valor.
•Definilo en el contrato y sé consistente (p. ej., JSON Schema).

4) Estructura y profundidad

•Evitá anidación profunda (> 4-5 niveles). Divide en sub-recursos o referencias (clienteId).
•Arrays: usalos para colecciones homogéneas; si mezclás tipos, justificá y documentá.
•Orden de claves: no es significativo; mantener orden lógico ayuda a diffs/revisiones.

5) Tamaño, paginación y filtros (APIs)

•Paginación: page, limit y metadatos (total, pages).

```
{ "data": [...], "pagination": { "page": 1, "limit": 20, "total": 347 } }
```

•Filtros y orden por query params (?search=teclado&sort=precio:desc).
•Límites de tamaño; compresión GZIP/Brotli.

6) Envoltorios de respuesta

Elegí un estilo y mantenelo:
•Plano: devolver directamente el recurso o lista.
•Envoltorio: data + meta/pagination/errors. Coherencia = clientes más simples.

7) Errores estructurados

Recomendado Problem Details (RFC 7807):

```
{
  "type": "https://api.ejemplo.com/errors/validation",
  "title": "Error de validación",
  "status": 400,
  "detail": "precio debe ser > 0",
  "errors": { "precio": ["Debe ser mayor a 0"] }
}
```

8) Versionado y compatibilidad

•Versioná la API: /v1/ o content-negotiation (Accept: application/vnd.miapi.v2+json).

•Compatibilidad hacia atrás: agregar campos debería ser tolerado.

•Remociones/cambios: período de deprecación + comunicación.

9) Validación con JSON Schema

•Definí types, required, rangos, pattern, enum.

•Validá requests y opcionalmente responses; versioná esquemas y usá contract tests.

10) Serialización estable

•Pretty-print solo para logs/dev; producción compacto.

•Para diffs determinísticos, usá serialización canónica (claves ordenadas).

•Formato de fechas único y zona (ideal UTC).

11) Seguridad

•Content-Type correcto: application/json; charset=utf-8.

•Tamaño máximo de payload; rechazá cuerpos gigantes.

•Sanitización: validá todo input; no confíes en el cliente.

•Redacción de PII/secretos en logs.

•CORS configurado.

•Rate limiting / idempotency-key en POST sensibles.

•Evitá JSON hijacking y CSRF: encabezados adecuados y autenticación fuerte.

12) Internacionalización y texto

•Siempre UTF-8.

•Evitá escapar Unicode innecesariamente (p. ej., opciones como JSON_UNESCAPED_UNICODE).

13) Comentarios y documentación

•JSON no admite comentarios. Si necesitás anotaciones, agregá campos explícitos (p. ej. _meta) o documentá en OpenAPI/Swagger.

•Proveé ejemplos válidos y casos de error en la documentación.

14) Actualizaciones parciales

•Merge Patch (application/merge-patch+json) para cambios simples:

```
{ "precio": 1399.99 }
```

•JSON Patch (application/json-patch+json) para operaciones declarativas:

```
[{ "op": "replace", "path": "/precio", "value": 1399.99 }]
```

15) Rendimiento

•Compresión en tránsito (GZIP/Brotli).

•Solo campos necesarios (proyecciones).

•NDJSON para streaming de muchos registros.

•En BDs relacionales: columnas virtuales + índices sobre propiedades JSON.

16) Ejemplo de contrato sano

```
{
  "id": 101,
  "descripcion": "Teclado mecánico",
  "precioArs": 1200.50,
  "disponible": true,
  "tags": ["periferico", "oferta"],
  "creadoEn": "2025-09-09T16:30:00Z",
  "meta": {
    "moneda": "ARS",
    "fuente": "catalogo"
  }
}
```

Notas: nombres consistentes, unidades explícitas, fecha ISO-8601 en UTC; meta para datos operativos que el cliente puede ignorar.

17) Checklist rápido

•Naming consistente (camelCase/snake_case), sin abreviaturas crípticas.

•Fechas ISO-8601 en UTC.

•Dinero como decimal/entero (centavos), no float.

•null vs ausente definidas.

•Validación con JSON Schema.

•Errores estructurados (RFC 7807).

•Paginación y filtros en listados.

•Tamaño y compresión controlados.

•Seguridad: headers, rate limit, redacción de logs.

•Versionado y compatibilidad hacia atrás.

18) Antipatrones comunes (y cómo corregirlos)

•Booleans como string: "activo": "true" → true.

•Fechas sueltas: "09/09/25" → "2025-09-09T00:00:00Z".

•Dinero en float: errores de coma flotante; usar decimal o centavos enteros.

•Campos sobrecargados: dividir en estado + subestado o motivo.

•Respuestas inconsistentes entre endpoints: unificar envoltorio, nombres y formatos.

•Anidación excesiva: normalizar o referenciar por id.

19) Herramientas útiles

•Validadores: JSON Schema (AJV en JS, jsonschema en Python).

•Formatters/linters: Prettier, VS Code, jq.

•Documentación: OpenAPI/Swagger con ejemplos request/response.

•Pruebas: contract testing (p. ej. Pact), tests de esquema en CI.

Aplicando estas prácticas obtenés JSON predecible, seguro, eficiente y fácil de mantener, reduciendo errores de integración y facilitando la evolución de tus APIs y servicios.